KENNESAW STATE UNIVERSITY

CS 4277

Diffusion Models

Dr. Chen Zhao

czhao4@kennesaw.edu

Computer Science

College of Computing and Software Engineering

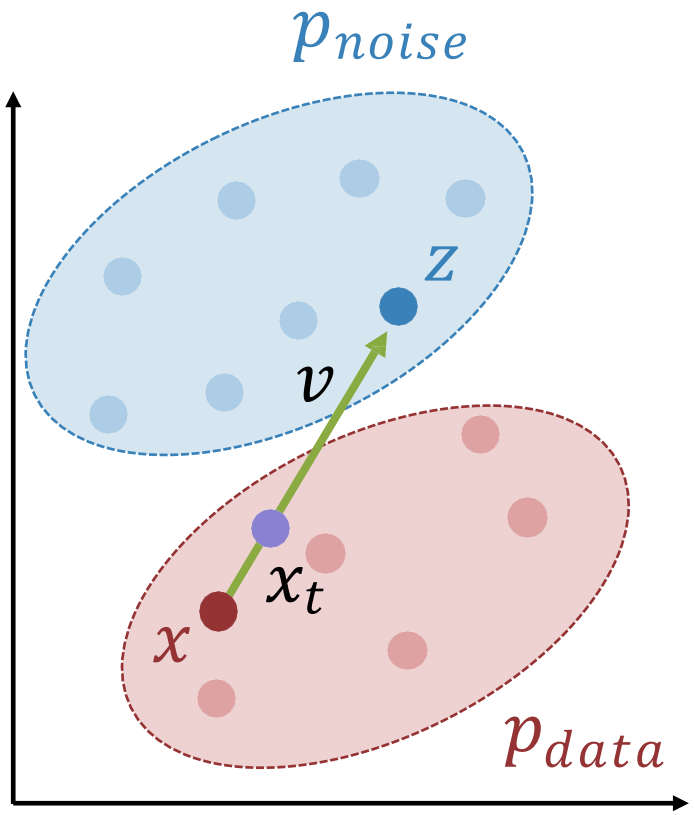
Learning Objectives

- Diffusion as Rectified Flow
- Diffusion Model in Math
- Learning
- Applications

Diffusion as Rectified Flow

Rectified Flow

- Suppose we have a simple p_{noise} (e.g. Gaussian) and samples from p_{data}
- On each training iteration, sample:
  - $z \sim p_{noise}$
  - $x \sim p_{data}$
  - $t \sim Uniform[0,1]$
- Set $x_t = (1-t)x + tz, v = z - x$
- This is the trace from data to noise
- v denotes the 'velocity' that transfers data to noise
- Train a neural network to predict v: $L = ||f_\theta(x_t, t) - v||_2^2$

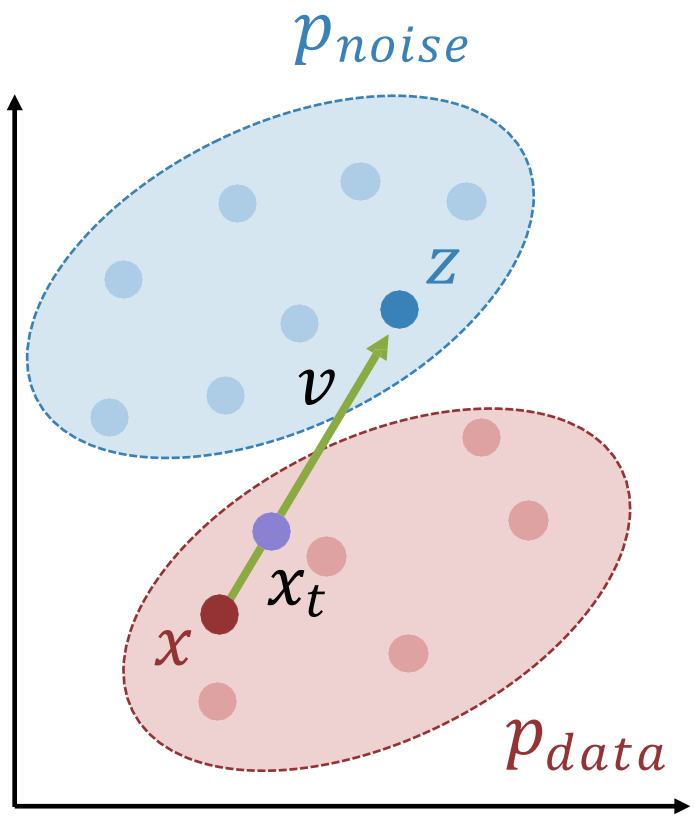


Liu et al, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow", 2022
Lipman et al, "Flow Matching for Generative Modeling", 2022

Diffusion Models: Rectified Flow

- Core training loop is just a few lines of code!

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

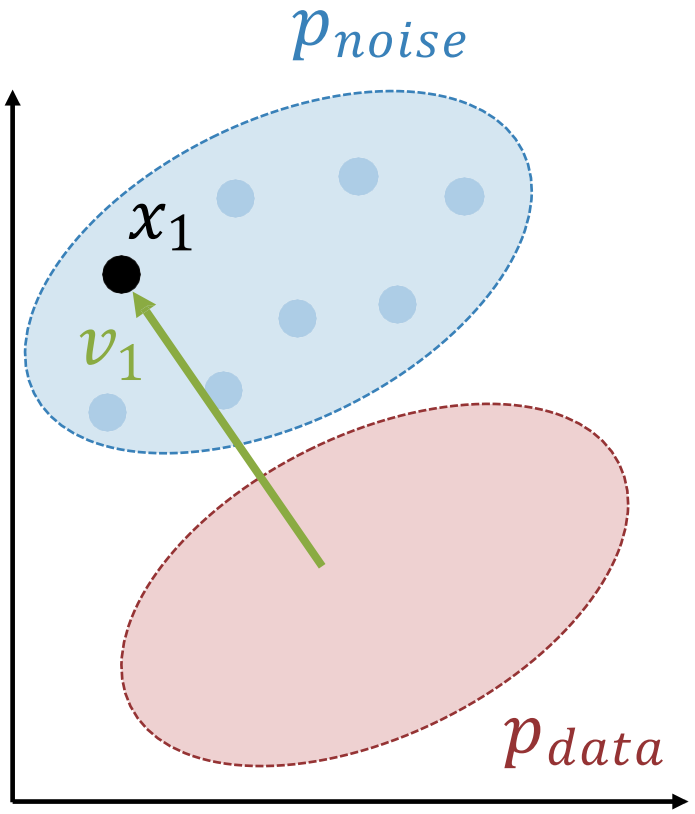


v: velocity predicted by network z: random noise x: data z-x: ground truth velocity

Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

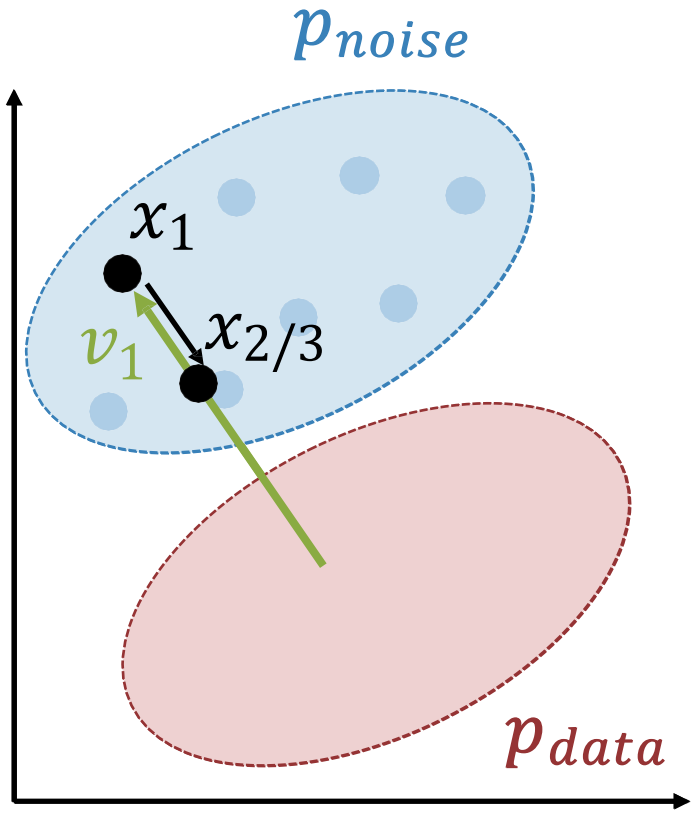
- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

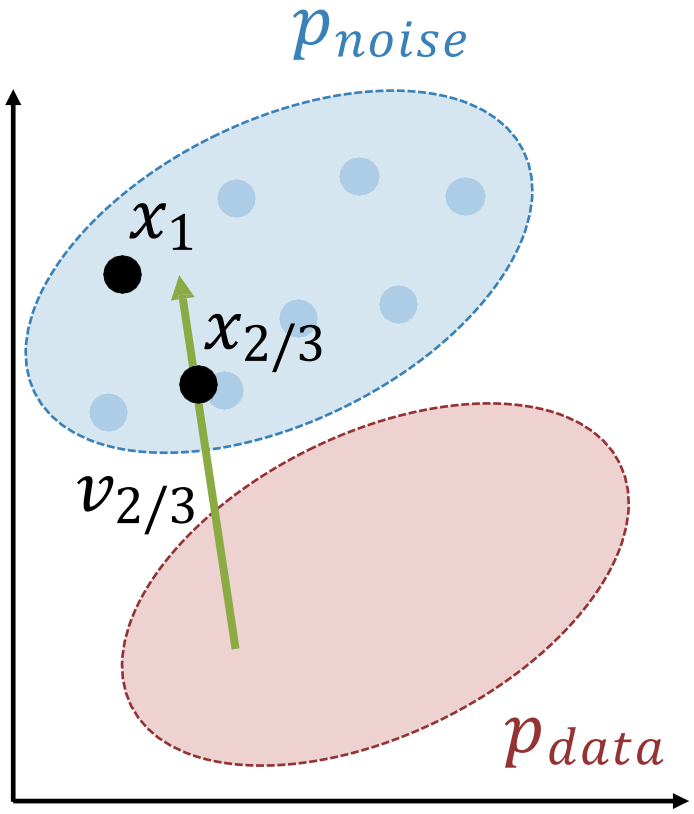
- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$
  - Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

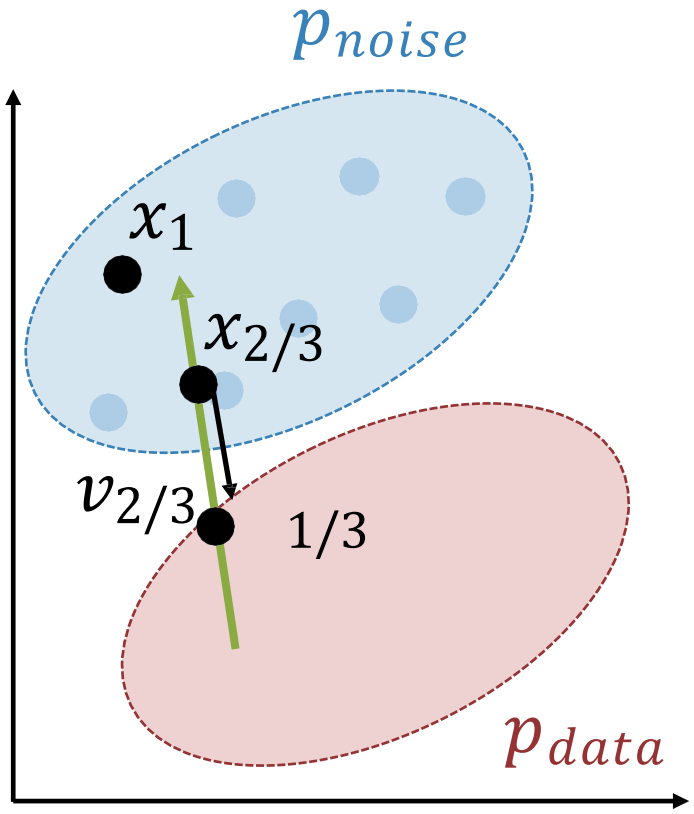
- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$
  - Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

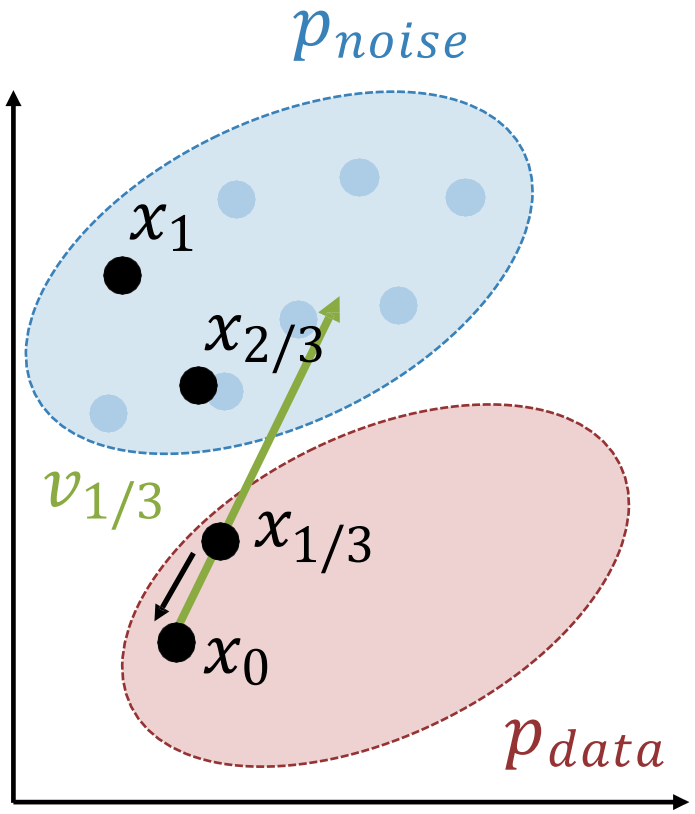
- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$
  - Step $x = x - v_t/T$



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$
  - Step $x = x - v_t/T$

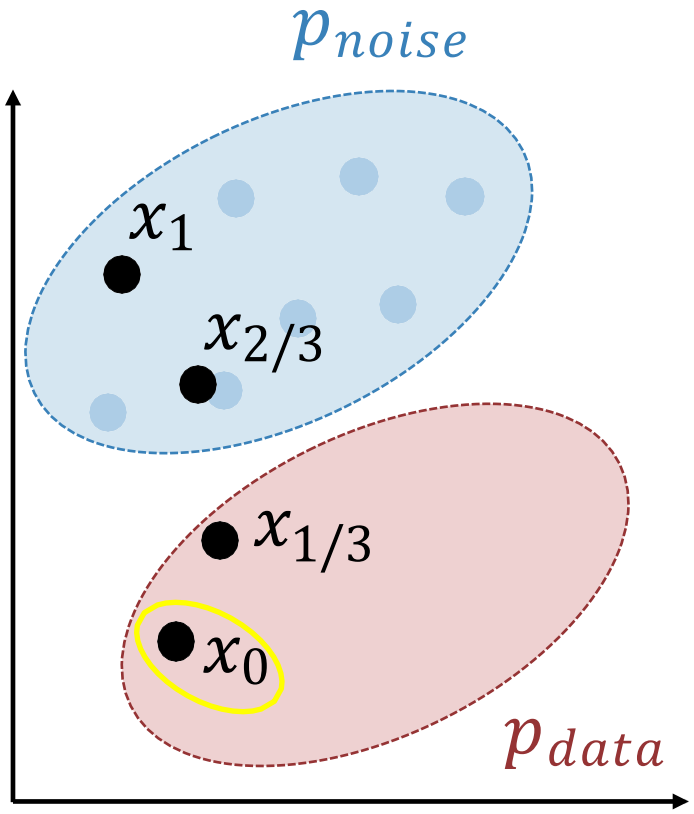


Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Rectified Flow: Sampling

- Choose number of steps T (often T=50)
- simple $x \sim p_{noise}$
- For t in $[1, 1-\frac{1}{T}, 1-\frac{2}{T}, \cdots, 0]$:
  - Evaluate $v_t = f_\theta(x_t, t)$
  - Step $x = x - v_t/T$
- Return x

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

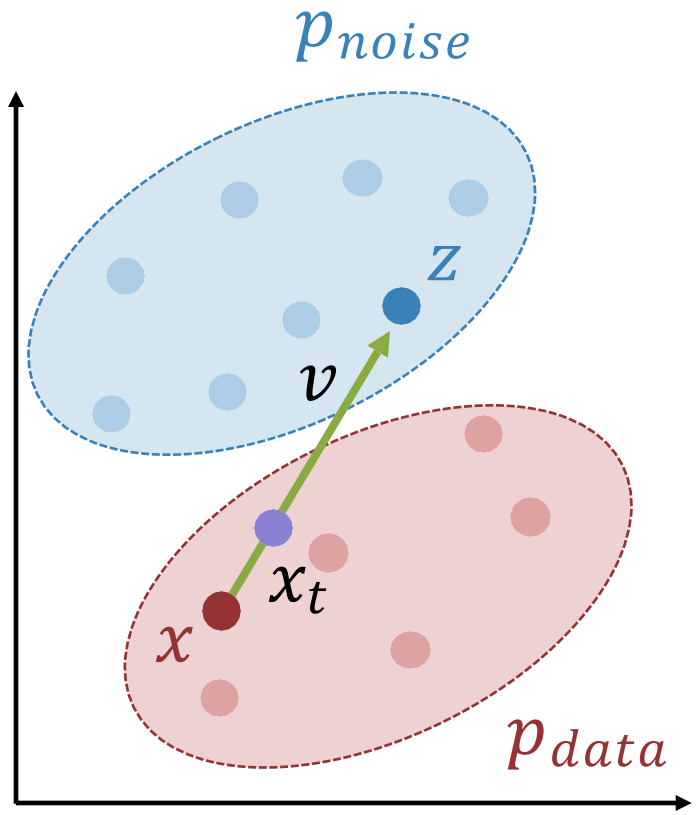
Rectified Flow: Summary

Training

```
for x in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, t)
    sample = sample - v / num_steps
```



Liu et al, “Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow”, 2022
Lipman et al, “Flow Matching for Generative Modeling”, 2022

Conditional Rectified Flow

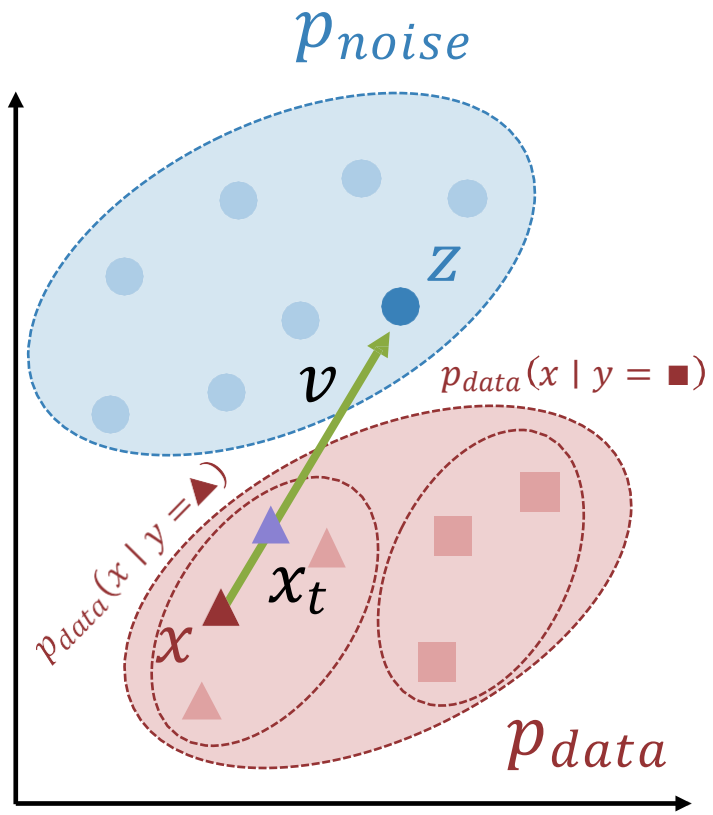
- Can we control how much we “emphasize” the conditioning y?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    v = model(sample, y, t)
    sample = sample - v / num_steps
```



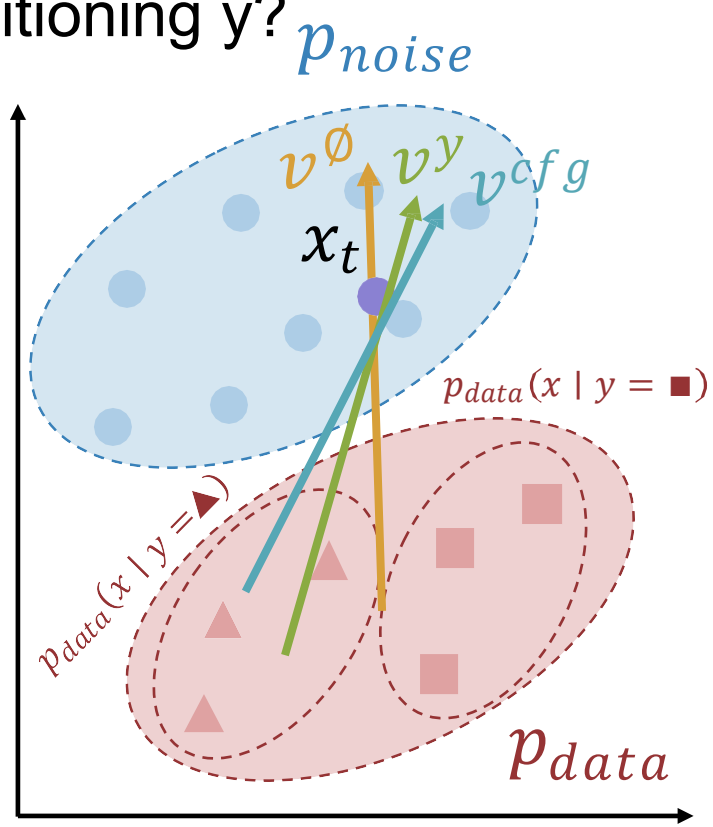
Classifier-Free Guidance (CFG)

- Can we control how much we “emphasize” the conditioning y?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

- Randomly drop y during training.
- Now the same model is conditional and unconditional!
- Consider a noisy x_t:
  - $v^{\emptyset} = f_\theta(x_t, y_\emptyset, t)$ points toward $p(x)$
  - $v^{y} = f_\theta(xt, y, t)$ points toward $p(x|y)$
  - $v^{cfg} = (1+w)v^{y} - wv^{\emptyset}$ points more toward $p(x|y)$
  - Classifier-Free Guidance pushes the diffusion sampling direction closer to the conditional distribution $p(x \mid y)$ by amplifying the difference between the conditional and unconditional denoising directions.



Classifier-Free Guidance (CFG)

- Can we control how much we “emphasize” the conditioning y?

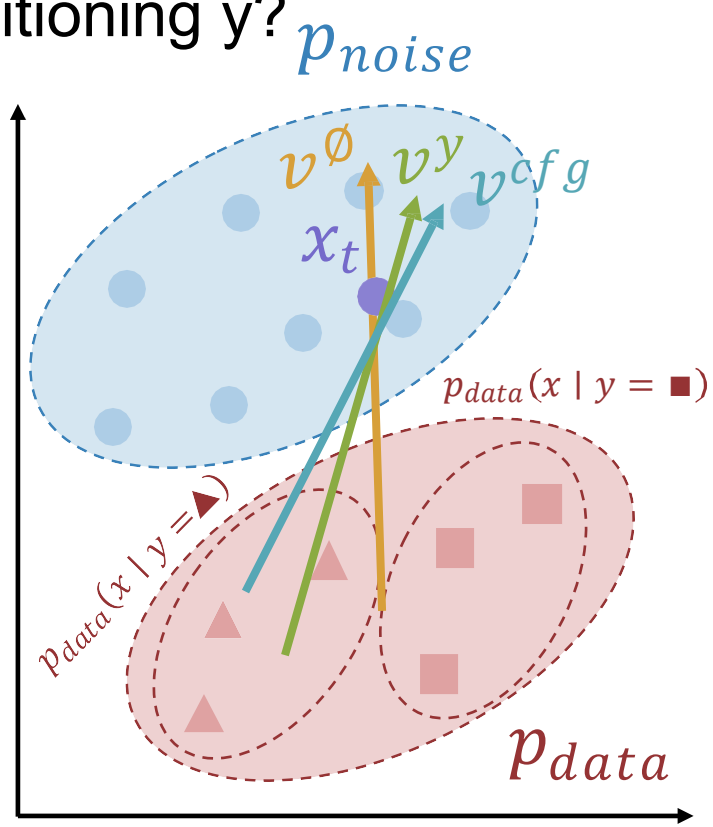
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

Sampling

```
y = user_input()
sample = torch.randn(x_shape)
for t in torch.linspace(1, 0, num_steps):
    vy = model(sample, y, t)
    v0 = model(sample, y_null, t)
    v = (1 + w) * vy - w * v0
    sample = sample - v / num_steps
```

Used everywhere in practice! Very important for high-quality outputs

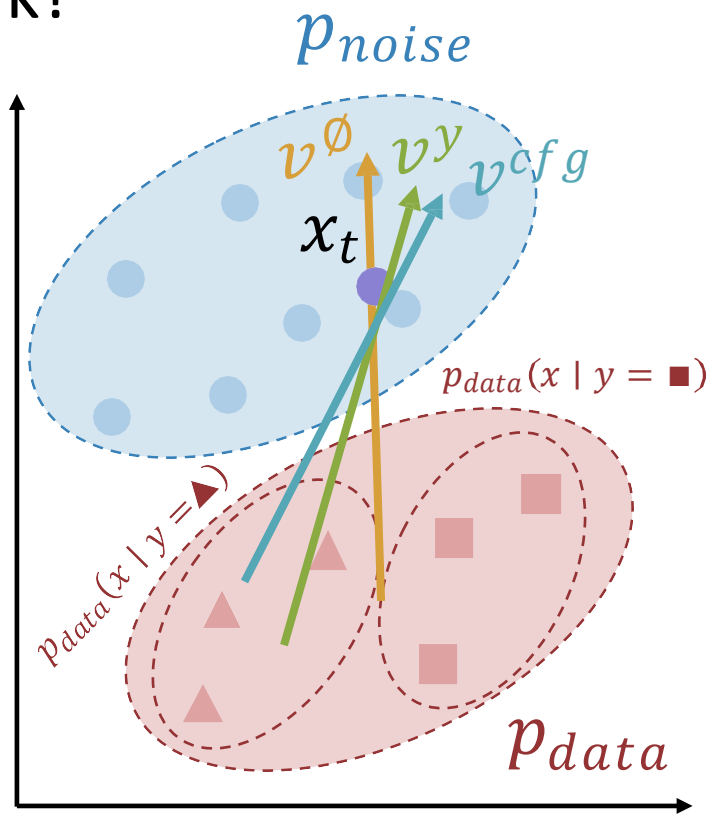


CFG: Optimal Prediction

- Q: What is the optimal prediction for the network?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```



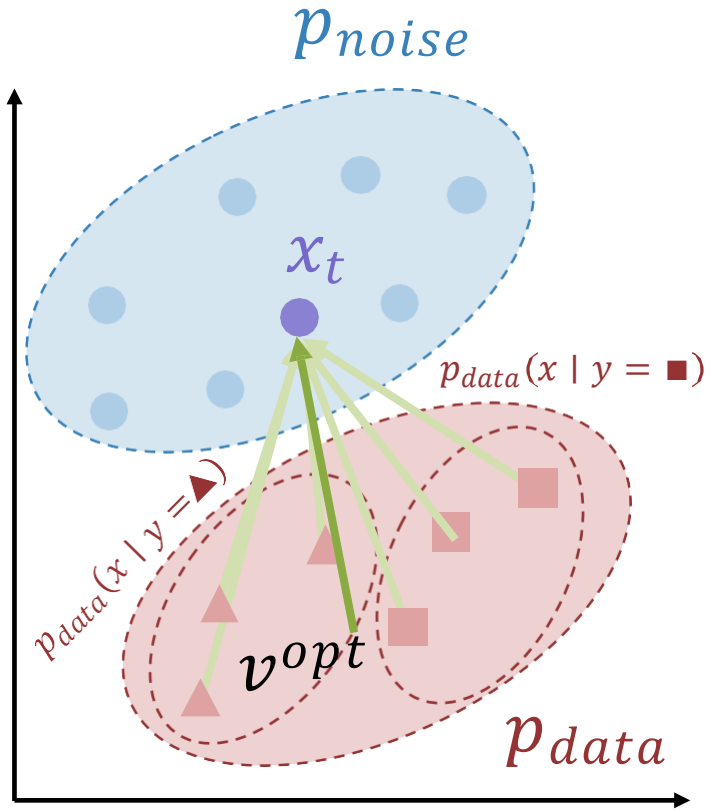
CFG: Optimal Prediction

- Q: What is the optimal prediction for the network?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

- There may be many pairs (x, z) that give the same x_t; network must average over them
- Full noise ($t = 1$) is easy: optimal v is mean of p_{data}



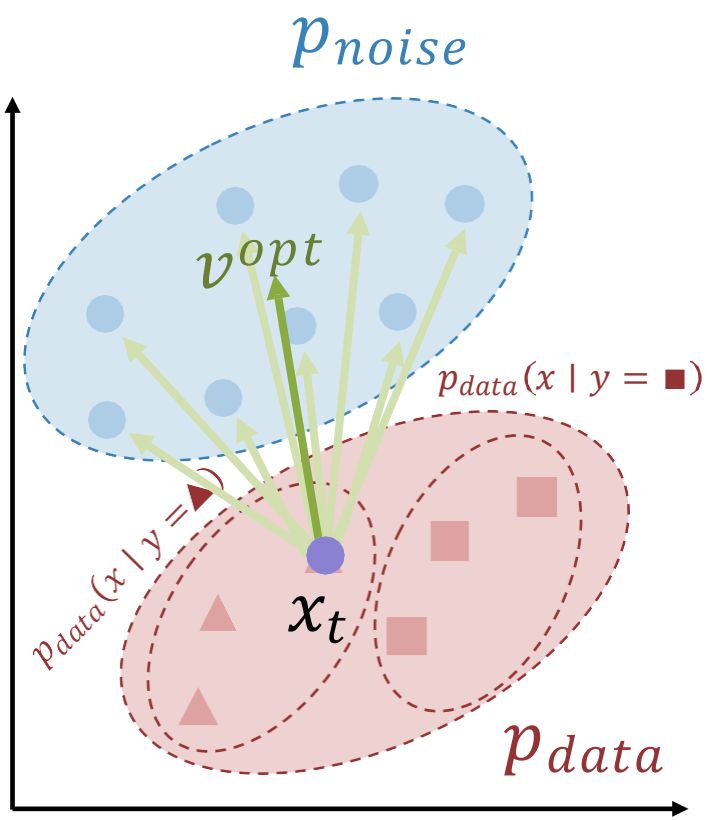
CFG: Optimal Prediction

- Q: What is the optimal prediction for the network?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

- There may be many pairs (x, z) that give the same x_t; network must average over them
- Full noise ($t = 1$) is easy: optimal v is mean of p_{data}
- No noise ($t = 0$) is easy: optimal v is mean of p_{noise}



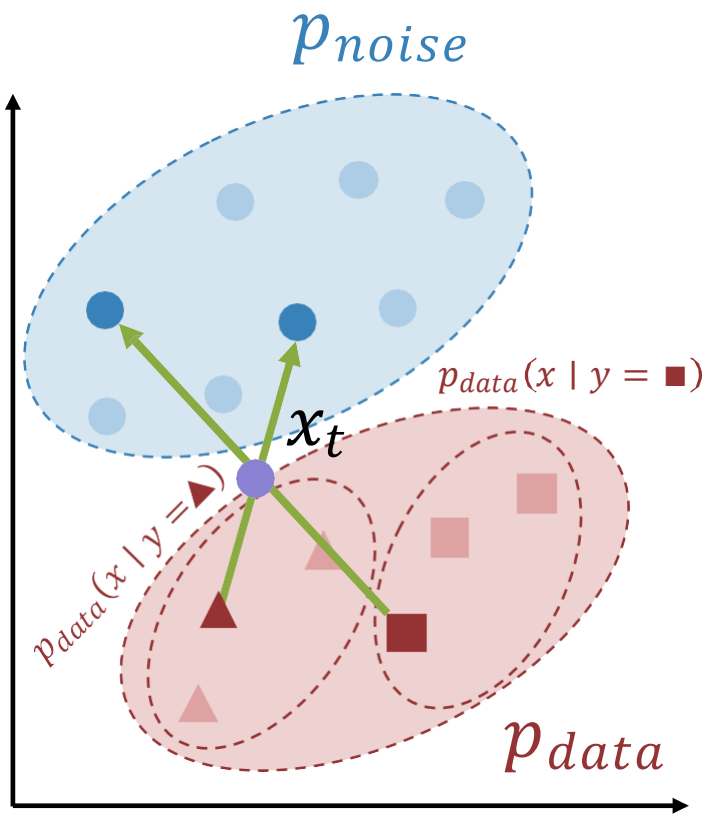
CFG: Optimal Prediction

- Q: What is the optimal prediction for the network?

Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = random.uniform(0, 1)
    xt = (1 - t) * x + t * z
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

- There may be many pairs (x, z) that give the same x_t; network must average over them
- Full noise ($t = 1$) is easy: optimal v is mean of p_{data}
- No noise ($t = 0$) is easy: optimal v is mean of p_{noise}
- Middle noise is hardest, most ambiguous
- But we give equal weight to all noise levels!
- Solution: Use a non-uniform noise schedule

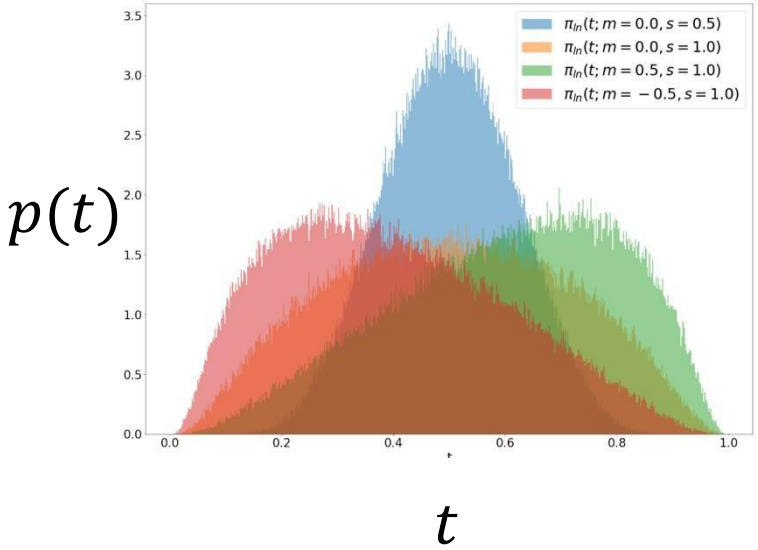


CFG: Optimal Prediction

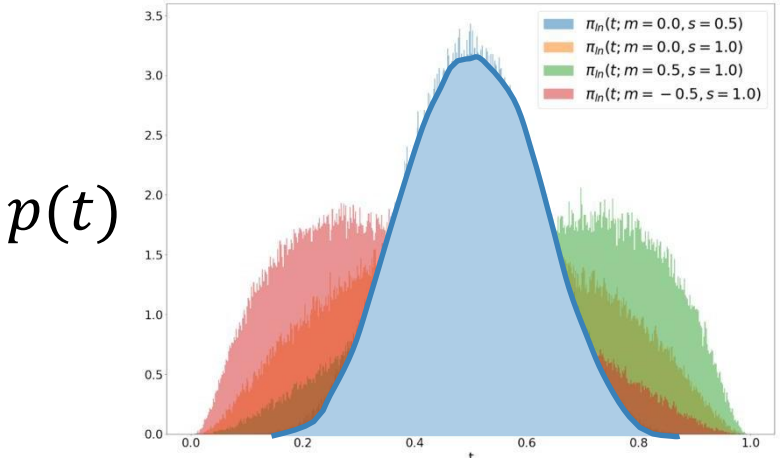
Training

```
for (x, y) in dataset:
    z = torch.randn_like(x)
    t = torch.randn(()).sigmoid()
    xt = (1 - t) * x + t * z
    if random.random() < 0.5: y = y_null
    v = model(xt, y, t)
    loss = (z - x - v).square().sum()
```

- There may be many pairs (x, z) that give the same x_t; network must average over them
- Full noise ($t = 1$) is easy: optimal v is mean of p_{data}
- No noise ($t = 0$) is easy: optimal v is mean of p_{noise}
- Middle noise is hardest, most ambiguous
- But we give equal weight to all noise levels!
- Solution: Use a non-uniform noise schedule

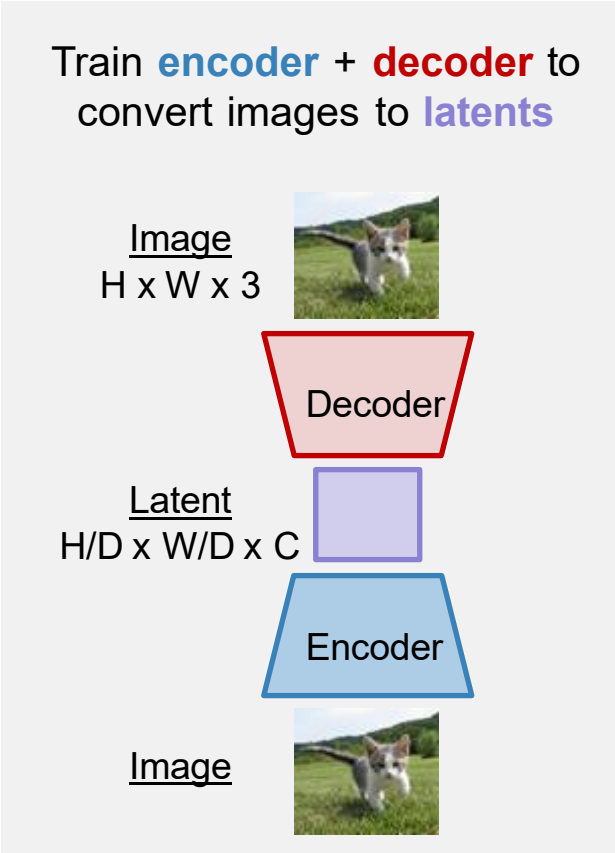


Put more emphasis on middle noise



Diffusion Model in Math

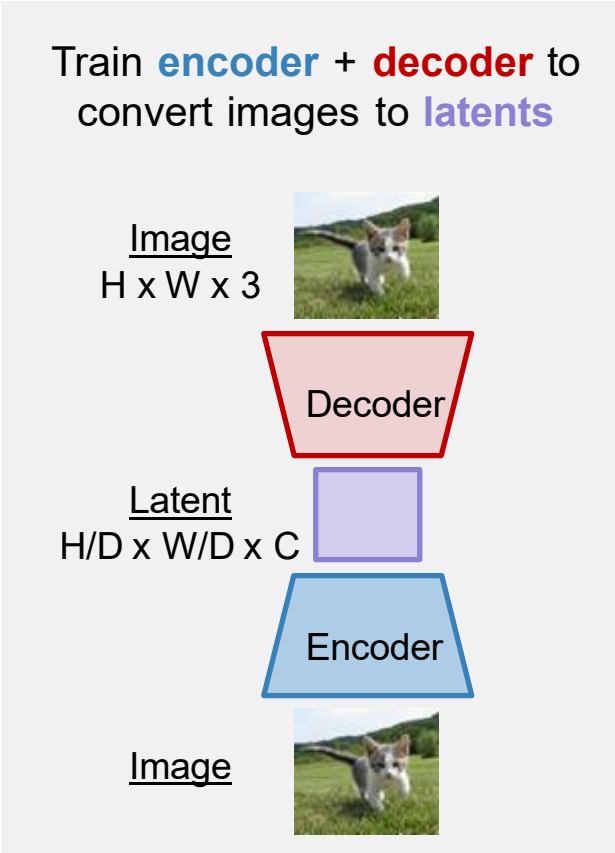
Latent Diffusion Models (LDMs)

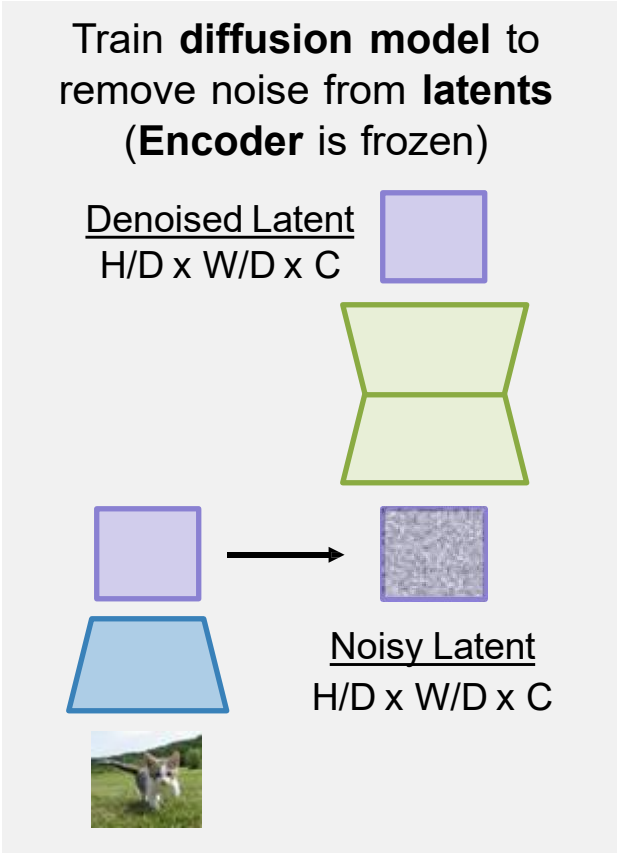


Common setting: D=8, C=16
Image: 256 x 256 x 3=> Latent: 32 x 32 x 16
Encoder / Decoder are CNNs with attention

Rombach et al, “High-Resolution Image Synthesis with Latent Diffusion Models”, CVPR 2022

Latent Diffusion Models (LDMs)

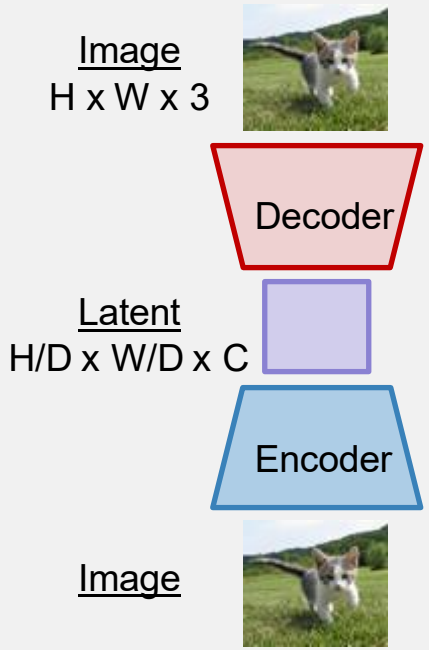




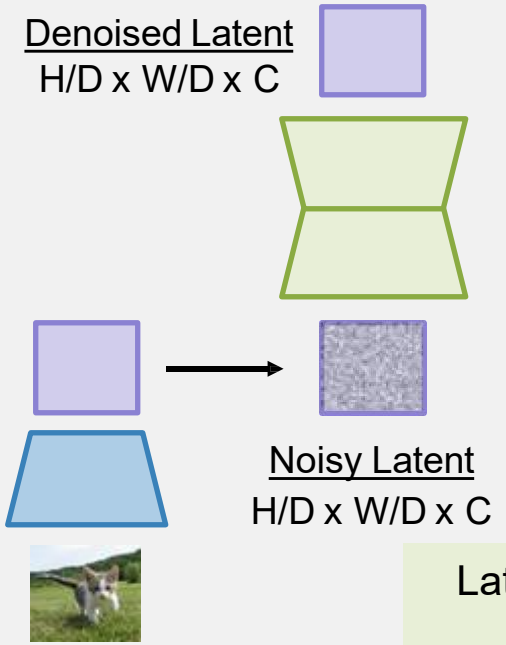
Rombach et al, “High-Resolution Image Synthesis with Latent Diffusion Models”, CVPR 2022

Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to convert images to **latents**



Train **diffusion model** to remove noise from **latents** (**Encoder** is frozen)

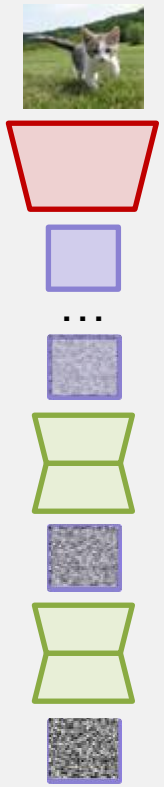


After training:

Sample random **latent**

Iteratively apply **diffusion model** to remove noise

run **decoder** to get **image**

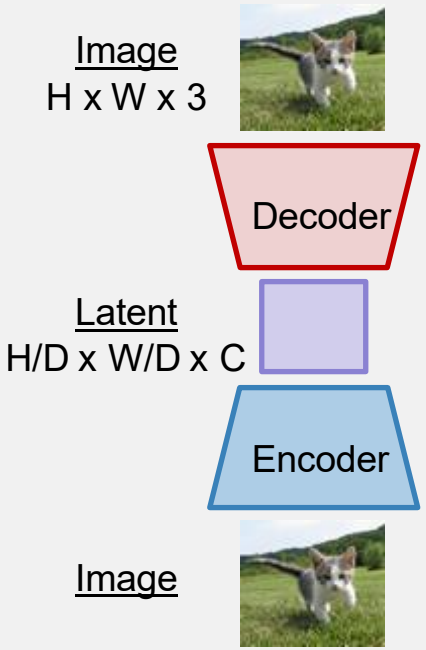


Latent diffusion is the most common form today

Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

Latent Diffusion Models (LDMs)

Train **encoder** + **decoder** to convert images to **latents**



How do we train the encoder+decoder?

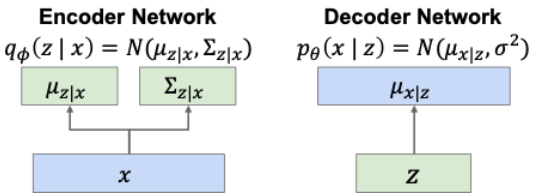
Solution: It's a VAE! Typically with very small KL prior weight

Problem: Decoder outputs often blurry
Solution: Add a discriminator!

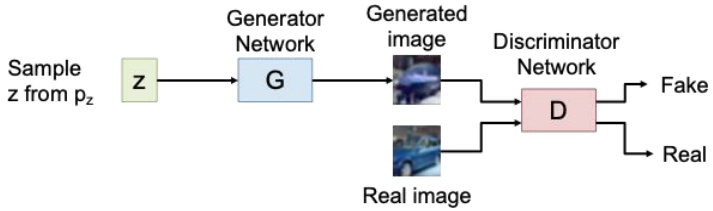
Morden LDM pipelines use VAE+GAN+diffusion

Recall: VAE

$$\log p_\theta(x) \geq E_{z\sim q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\left(q_\phi(z|x), p(z)\right)$$



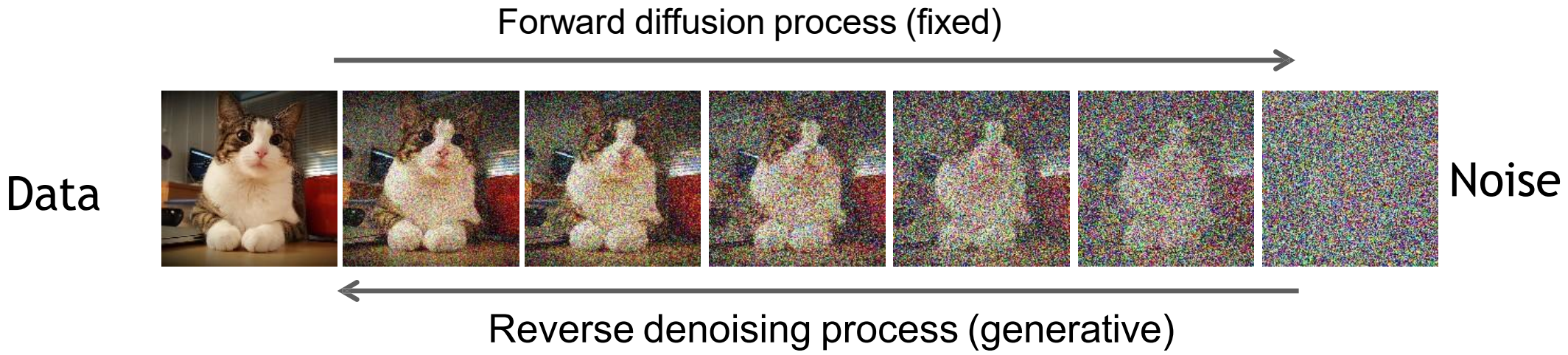
Recall: GAN



Rombach et al, "High-Resolution Image Synthesis with Latent Diffusion Models", CVPR 2022

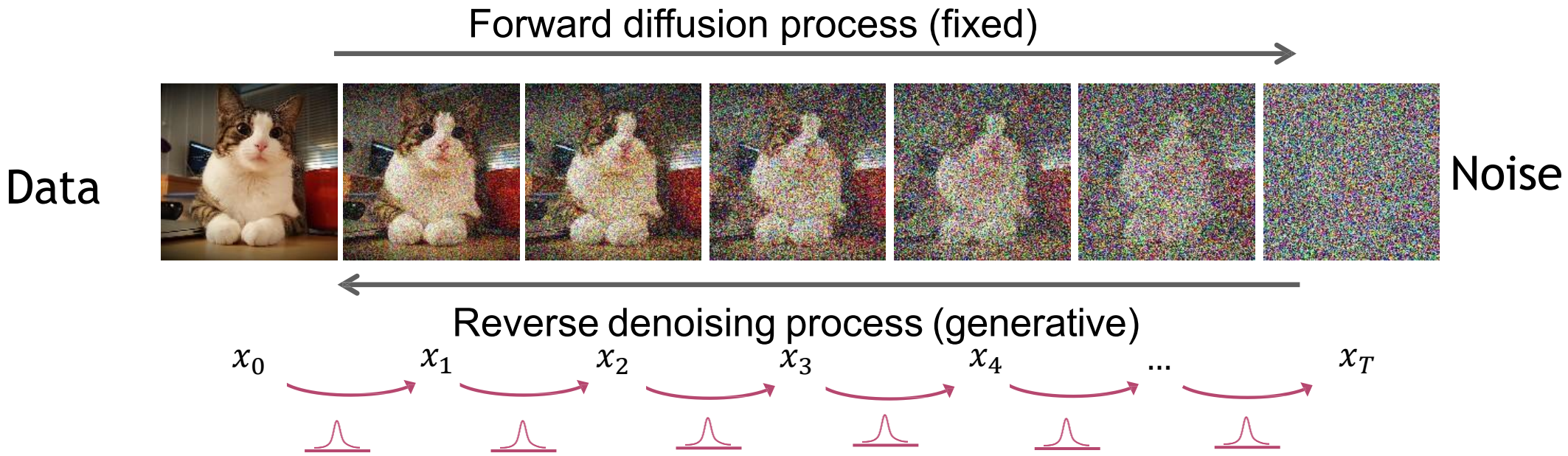
Denoising Diffusion Models

- Function: Learning to generate by denoising
- Denoising diffusion models consist of two processes:
  - Forward diffusion process that gradually adds noise to input
  - Reverse denoising process that learns to generate data by denoising



Forward Diffusion Process

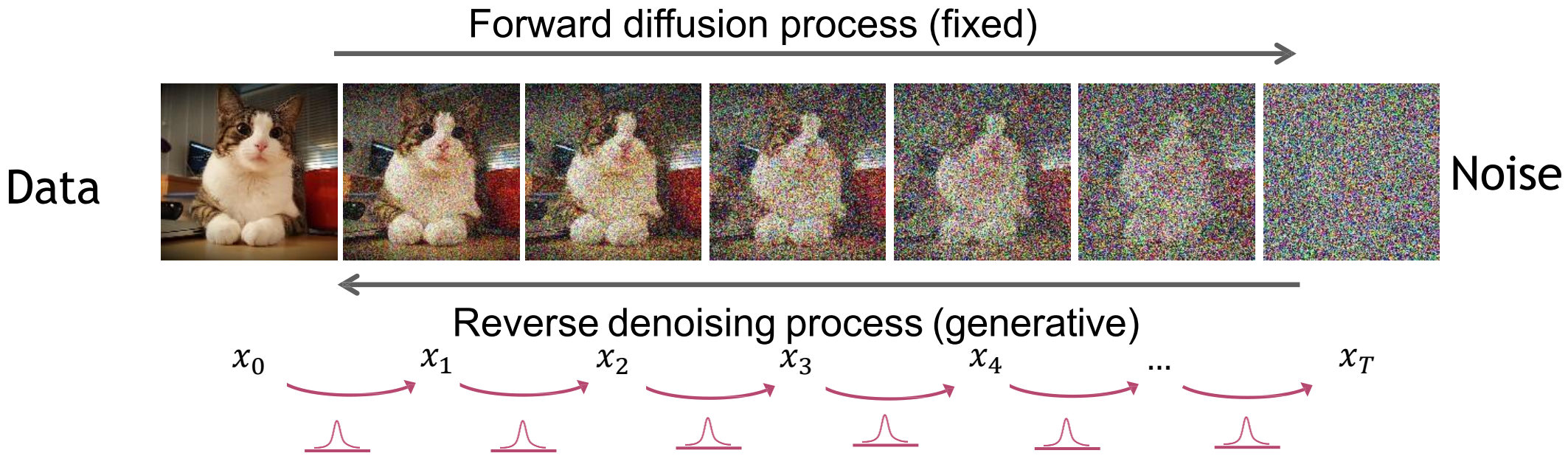
- The formal definition of the forward process in T steps:



- $q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$ => $q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$
- Similar to the inference model in hierarchical VAEs

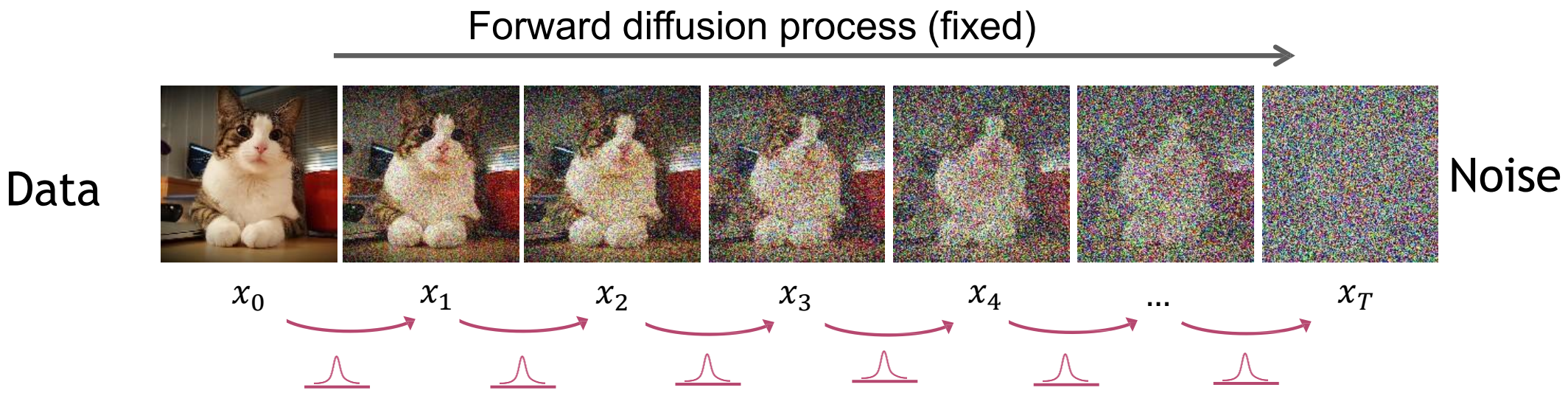
Forward Diffusion Process

- The formal definition of the forward process in T steps:



- $q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$ => $q(x_{1:T}|x_0) = \prod_{t=1}^{T} q(x_t|x_{t-1})$
- Similar to the inference model in hierarchical VAEs

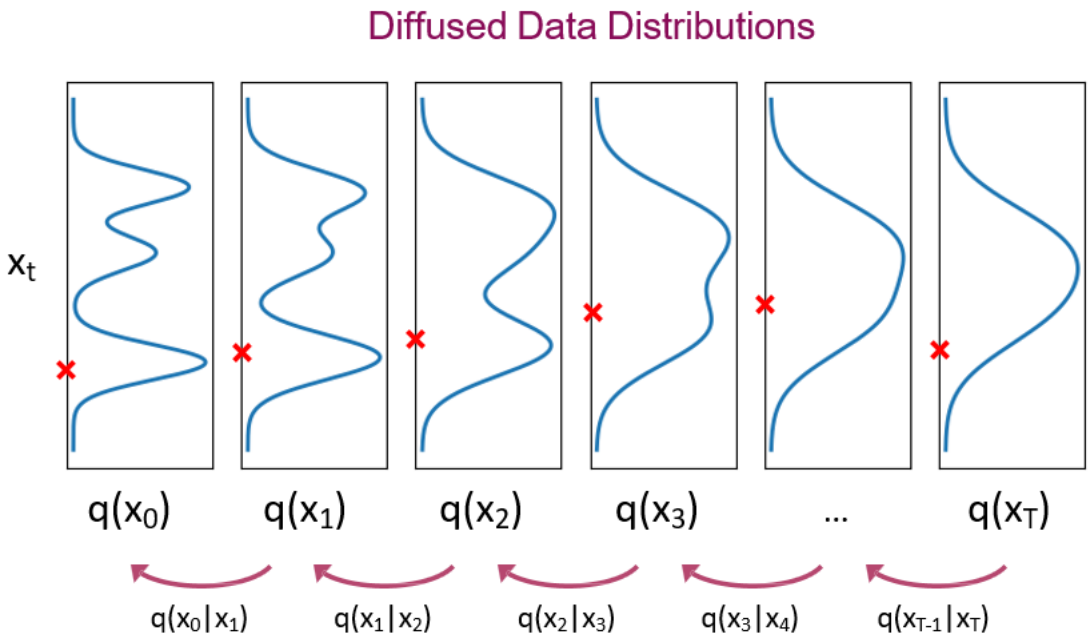
Diffusion Kernel



- Define $\bar{\alpha}_t = \prod_{s=1}^{T}(1-\beta_s)$
- $q(x_t|x_0) = N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ (Diffusion Kernel)
- For sampling $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{(1-\bar{\alpha}_t)}\epsilon$, where $\epsilon \sim N(0, I)$
- β_t values schedule is designed such that $\bar{\alpha}_t \rightarrow 0$ and $q(x_T|x_0) \approx N(x_T; 0, I)$

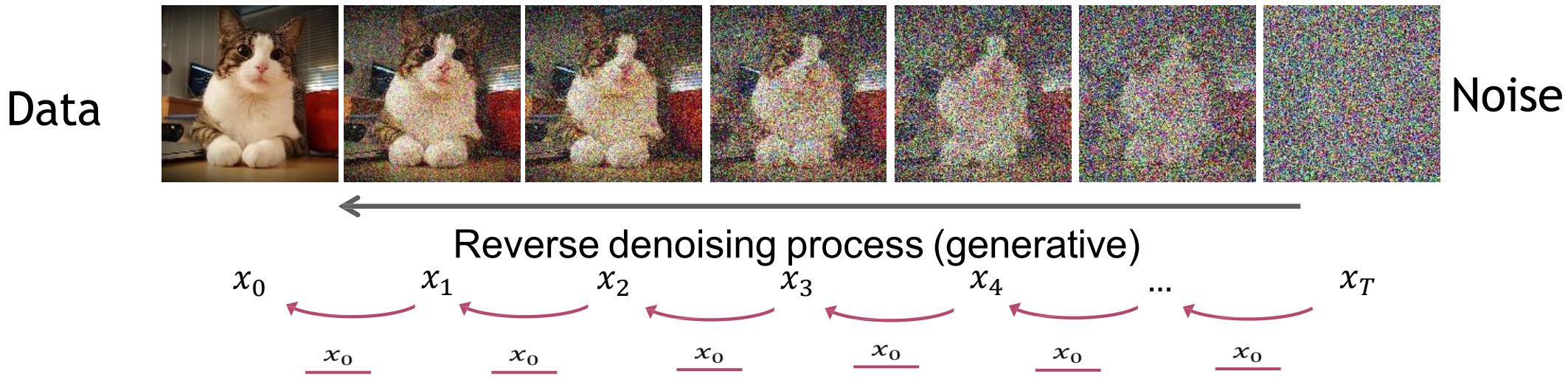
Generative Learning by Denoising

- Recall, that the diffusion parameters are designed such that $q(x_T) \approx N(x_T; 0, I)$
- Generation:
- Sample $x_T \sim N(x_T; 0, I)$
- Iteratively sample $x_{t-1} \sim q(x_{t-1}|x_t)$
- In general, $q(x_{t-1}|x_t) \propto q(x_{t-1})q(x_t|x_{t-1})$ is intractable.
- Can we approximate $q(x_{t-1}|x_t)$?
- Yes, we can use a Gaussian distribution if β_t is small in each forward diffusion step



Reverse Denoising Process

- Formal definition of forward and reverse process in T steps.



- $p(x_T) = N(x_T; 0, I)$;
- $p(x_{t-1}|x_t) = N(x_{t-1}; \mu_\theta(x_t, t), \sigma_t^2 I)$ (Trainable network, U-Net)
- $p_\theta(x_{0;T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$
- Similar to the generative model in hierarchical VAEs.

Learning

Learning Denoising Model

- How? Variational upper bound
- For training, we can form variational upper bound (negative ELBO) that is commonly used for training variational autoencoders.

$$E_{q(x_0)}[-\log p_\theta(x_0)] \leq E_{q(x_0)q(x_{1:T}|x_0)}[-\log \frac{p_\theta(x_{0:T})}{q(x_{1:T}|x_0)}]$$

- For diffusion model

$$L = E_q\left[D_{KL}\big(q(x_T|x_0), p(x_T)\big) + \sum_{t>1} D_{KL}\big(q(x_{t-1}|x_t, x_0), p(x_{t-1}|x_t)\big) - \log p_\theta(x_0|x_1)\right]$$

$$L = E_q\left[L_T + \sum_{t>1} L_{t-1} - L_0\right]$$

- where $q(x_{t-1}|x_t, x_0)$ is the tractable posterior distribution that

$$q(x_{t-1}|x_t, x_0) = N(x_{t-1}; \bar{\mu}_t(x_t, x_0), \beta_t I)$$

- where $\bar{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{1-\beta_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t$ and $\bar{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$

Learning Denoising Model

$$L = E_q\left[D_{KL}\big(q(x_T|x_0), p(x_T)\big) + \sum_{t>1} D_{KL}\big(q(x_{t-1}|x_t, x_0), p(x_{t-1}|x_t)\big) - \log p_\theta(x_0|x_1)\right]$$

- Since both $q(x_{t-1}|x_t, x_0)$ and $p_\theta(x_{t-1}|x_t)$ are normal distribution, the KL divergence has a simple form that

$$L_{t-1} = D_{KL}\big(q(x_{t-1}|x_t, x_0), p(x_{t-1}|x_t)\big) = E_q\left[\frac{1}{2\sigma_t^2}\left|\left|\bar{\mu}_t(x_t, x_0) - \mu_\theta(x_t, t)\right|\right|^2\right] + C$$

- Recall that $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, then $\bar{\mu}_t(x_t, x_0) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon\right)$
- They propose to represent the mean of the denosing model using a noise-prediction network
- $\mu_\theta(x_t, t) = \frac{1}{\sqrt{1-\beta_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon(x_t, t)\right)$
- $L_{t-1} = E_{x_0\sim q(x_0), \epsilon\sim N(0,I)}\left[\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}\left|\left|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)\right|\right|^2\right] + C$
- where $\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon = x_t$

Learning Denoising Model

- $L_{t-1} = E_{x_{0\sim q(x_0),\epsilon\sim N(0,I)}}\left[\frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}\left|\left|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)\right|\right|^2\right] + C$
- The time dependent $\lambda_t = \frac{\beta_t^2}{2\sigma_t^2(1-\beta_t)(1-\bar{\alpha}_t)}$ ensures that the training objective is weighted properly for the maximum data likelihood training.
- However, this weight is often very large for small t's.
- Ho et al. NeurIPS 2020 observe that simply setting $\lambda_t = 1$ improves sample quality.
- So, they propose to use:
- $L_{simple} = E_{x_{0\sim q(x_0),\epsilon\sim N(0,I),t\sim U(1,T)}}\left[\left|\left|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t\right)\right|\right|^2\right]$

Summary

- Training and Sample Generation

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \mathrm{Uniform}(\{1, \ldots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on

$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boxed{\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}}, t) \right\|^2$$

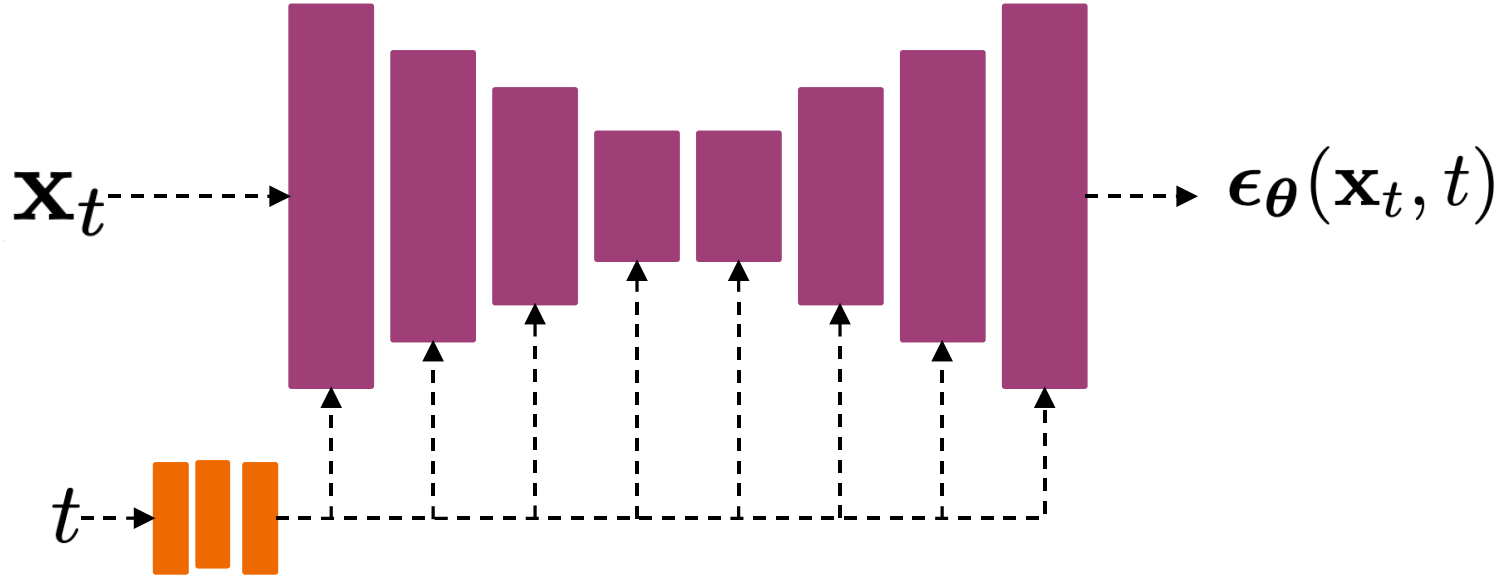
6: **until** converged

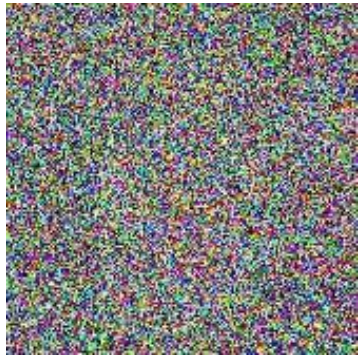
Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
4: $\mathbf{x}_{t-1} = \boxed{\frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right)} + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Implementation Considerations

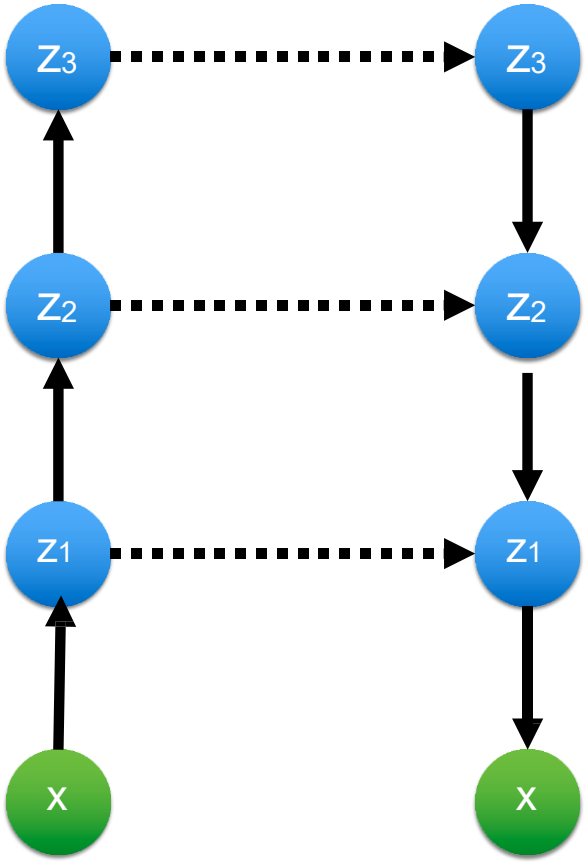
- Network Architectures
- Diffusion models often use U-Net architectures with ResNet blocks and self-attention layers to represent $\epsilon(x_t, t)$





Connection to VAEs

- Diffusion models can be considered as a special form of hierarchical VAEs.
- However, in diffusion models:
  - The inference model is fixed: easier to optimize
  - The latent variables have the same dimension as the data.
  - The ELOB is decomposed to each time step: fast to train
  - Can be made extremely deep (even infinitely deep)
  - The model is trained with some reweighting of the ELBO.



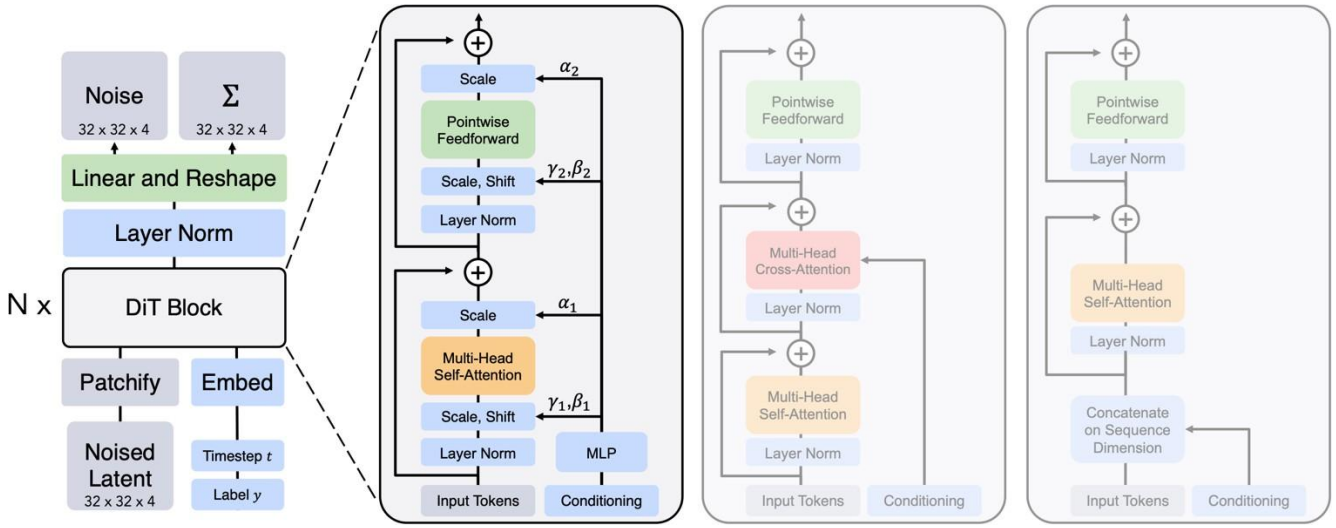
Inference model
q(z|x)

Generative model
p(x,z)

Applications

Diffusion Transformer (DiT)

- Diffusion uses standard Transformer blocks!
- Main question: How to inject conditioning (timestep t, text, …)



Predict scale/shift: Most common for diffusion timestep t

Cross-Attention / Joint Attention: Common for text, image, etc conditioning

Text-to-Image Generation

Diffusion timestep
(scalar)

Noisy latents
128 x 128 x 16

Text embeddings
D x L

Diffusion Transformer

Clean latents
128 x 128 x 16

Decoder

Output image
1024 x 1024 x 3

Pretrained text encoder
(e.g. T5, CLIP)

Text Prompt
A professional documentary photograph of a monkey shaking hands with a tiger in front of the Eiffel tower. The monkey is wearing a hat made out of bananas, and the tiger is standing on two legs and wearing a suit.

Example: FLUX.1 [dev]

Text Encoder: T5 + CLIP

Encoder/Decoder: 8x8 downsampling
Diffusion model: 12B parameter model

2x2 patchify => 64x64 = 1024 image tokens

FLUX

Text-to-Video Generation

Diffusion timestep
(scalar)

Noisy latents
32 x 128 x 72 x 16

Text embeddings
D x L

Diffusion Transformer

Clean latents
33 x 128 x 72 x 16

Decoder

Output video
257 x 1024 x 576 x 3

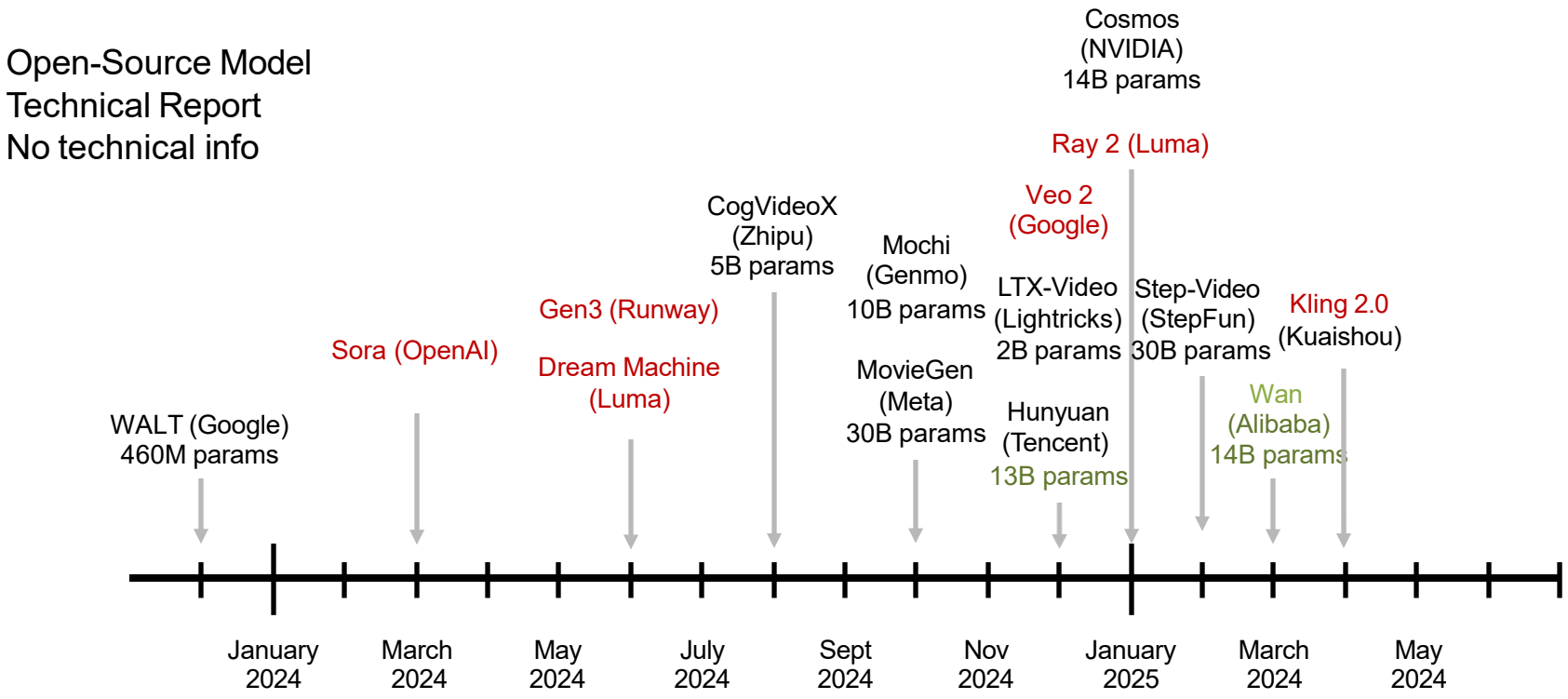
Pretrained text encoder
(e.g. T5, CLIP)

Text Prompt
A red-faced monkey with white fur is bathing in a natural hot spring. The monkey is playing in the water with a miniature sail ship in front of it, made of wood with a white sail and a small rudder. The hot spring is surrounded by lush greenery, with rocks and trees.

Example: Meta MovieGen

Text Encoder: UL2, ByT5, MetaCLIP
Encoder/Decoder: 8x8x8 downsample
Diffusion model: 30B param DiT 1x2x2 patchify => 76K tokens

The Era of Video Diffusion Models



References

- CS231n